DGB SIM

DGB SIM

Etapa 1 | Tabela Hash e Aplicação Web

- Bruno Barreto
- Dimitrius Khouri
- Guilherme Lopes



DGB SIM

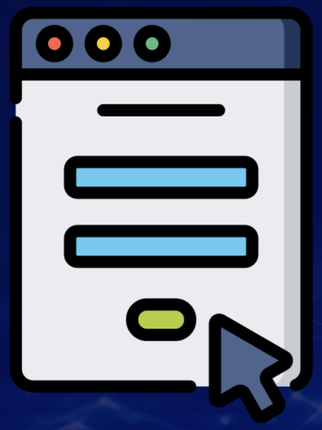
O problema e a escolha da estrutura

- Problema: consultar cobertura a partir do código do país.
- Chave externa: código como chave; dicionário com os dados do país como valor.
- TabelaHash organiza as operações e dict fornece o mecanismo de hash.
- JSON conserva os dados e Flask conecta a estrutura à interface.

DGB SIM

Como a consulta atravessa o sistema

Formulário



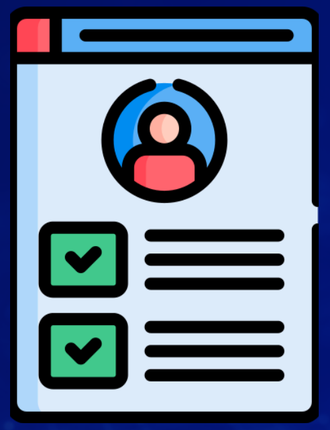
Rota Flask



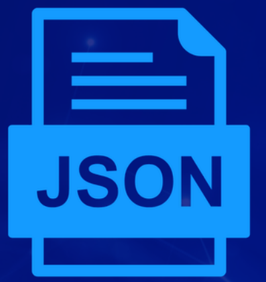
Tabela Hash



Resultado



JSON



JSON → Hash
Carga inicial

O formulário envia o código à rota Flask, que chama buscar() na Tabela Hash e encaminha o resultado à interface. O JSON carrega os dados na inicialização.

DGB SIM

Busca por chave: do código ao registro

O código é normalizado antes da busca.

get() consulta a chave e retorna os dados do país ou None, sem busca sequencial explícita.

O dict gerencia o hash e as colisões internamente.

```
# buscar um país pelo código.
@_medir("buscar", falha_se_none=True)
def buscar(self, codigo):
    # normaliza o código e usa .get() para consultar diretamente o dicionário
    codigo = codigo.strip().upper()

    return self.paises.get(codigo)
```

DGB SIM

Inserção, alteração e remoção

Inserir: valida os campos e rejeita chave duplicada.

```
# normaliza o código
@_medir("inserir")
def inserir(self, codigo, nome, regiao, operadora, tecnologia):
    codigo:str = codigo.strip().upper()

    # validação de duas letras
    if len(codigo) != 2 or not (codigo.isalpha() and codigo.isascii()):
      raise ValueError("O código do país deve possuir exatamente 2 letras.")

    #impede que escreva um código já existente
    if codigo in self.paises:
        raise ValueError("Já existe um país cadastrado com esse código")

    self._validar_dados(nome, regiao, operadora, tecnologia)

    self.paises[codigo] = {
        "nome": nome,
        "regiao": regiao,
        "operadora": operadora,
        "tecnologia": tecnologia
    }
```

DGB SIM

Inserção, alteração e remoção

Alterar: exige registro existente e preserva o código.

```
    # alterar os dados de um país existente, mantendo seu código.
    @_medir("alterar")
    def alterar(self, codigo, nome, regiao, operadora, tecnologia):
        codigo = codigo.strip().upper()

        #verificação impede que uma alteração cadastre acidentalmente um país
inexistente.
        if codigo not in self.paises:
            raise ValueError("País não encontrado.")

        self._validar_dados(nome, regiao, operadora, tecnologia)

        self.paises[codigo] = {
            "nome": nome,
            "regiao": regiao,
            "operadora": operadora,
            "tecnologia": tecnologia
        }
```

DGB SIM

Inserção, alteração e remoção

Remover: verifica a chave antes da exclusão.

```
# remover um país
@_medir("remover")
def remover(self, codigo):
    codigo = codigo.strip().upper()

    #verificação permite informar quando o país não existe.
    if codigo not in self.paises:
        raise ValueError("País não encontrado.")

    del self.paises[codigo]
```

Acrescenta-se que: após inserir, alterar ou remover com sucesso, a rota salva o JSON.

DGB SIM

Flask e JSON

Inicialização: JSON → carregar_paises() → TabelaHash

Consulta: Flask → buscar() → template Jinja

Alterações: POST → método da estrutura → salvar_paises()

A memória atende às operações; o arquivo preserva os registros entre execuções.

As rotas tratam a requisição; a classe concentra as operações e validações.





DGB SIM

Diferencial proposto: Roteamento multicritério com histórico de consultas

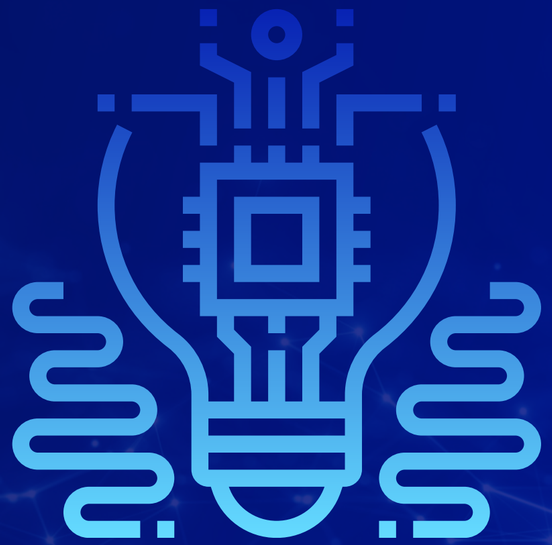
01 | Seleção do critério: Permitir ao usuário escolher entre menor latência e menor custo operacional simulado.

02 | Cálculo da rota: Aplicar o algoritmo de Dijkstra ao peso selecionado, determinando uma rota por consulta, sem otimização simultânea dos critérios.

03 | Apresentação do resultado: Exibir a sequência de países, o critério utilizado, a latência total e o custo operacional total do caminho.

04 | Histórico persistente: Registrar em JSON a origem, o destino, o critério, a rota e os totais obtidos, permitindo consultas posteriores.

05 | Validação das condições de cálculo: Rejeitar pesos negativos ou ausentes e informar localidades indisponíveis ou inexistência de caminho. Identificar os valores como simulados.



Obs.: Implementação prevista para a etapa de grafos.

DGB SIM

Conclusão

Aplicação da Tabela Hash ao gerenciamento dos países, utilizando o código como chave e o dicionário Python como base da estrutura.

Integração das operações e validações à aplicação Web, com persistência em JSON e instrumentação para acompanhar seu funcionamento.

Articulação entre conceitos de estruturas de dados, implementação e uso do sistema.

Continuidade planejada com Árvore B, grafos e a proposta de roteamento multicritério com histórico de consultas.